IGAME

igame 아키텍처 및 보안 백서 (Architecture & Security Whitepaper)

본 문서는 igame의 단일 바이너리 모듈러 모놀리스 아키텍처, 설치키 기반 공급자 비밀 암호화와 개인 API/MCP 키의 해시 저장·즉시 회전, Keycloak OIDC SSO 및 Model Context Protocol (MCP) 연동에 대한 기술 사양서입니다.

서비스 버전 v0.7.34
발행일 2026-10-16
Apache License 2.0 · 사내 폐쇄망 배포용

🏛️ 1. 시스템 토폴로지 및 런타임 구조

igame은 Go 1.26+ 기반의 단일 실행 바이너리 안에 React 19 정적 자산과 Phaser 엔진 번들을 내장(go:embed)하여, 외부 인터넷 통신이 일체 없는 폐쇄망(Air-Gapped) 환경에서 동작합니다.

```
Client Browser (React 19 + Phaser)
                | (HTTPS / Bearer Token / Session)
                v
+-------------------------------------------------------------+
| igame Modular Monolith (:8080)                              |
|  +- Core Control Plane (REST / SSE / MCP Streamable HTTP)   |
|  +- Keycloak OIDC SSO & Local Bootstrap Auth                |
|  +- Deterministic Battle Kernel & Replay Verifier           |
|  +- Game Session & Telemetry Ledger Validator               |
|  +- Leaderboard, Season & Tournament Engine                 |
|  +- OIDC/AI Secrets: Installation-Key AES-256-GCM           |
|  +- Personal API/MCP Keys: SHA-256 Verification Hashes      |
|  +- Embedded Web Assets (dist/*)                            |
+-------------------------------------------------------------+
                | (SQL / pgxpool)
                v
+-------------------------------------------------------------+
| PostgreSQL 16+ (ACID Transactional Data Store)              |
+-------------------------------------------------------------+
```

1.1 서버 권위 전투 검증 (Server-Authoritative Battle Replay)

RealmGuard의 전투 규칙은 renderer와 완전히 분리된 결정론적 kernel 하나로 존재하며, 브라우저(TypeScript)와 서버(Go)에 같은 알고리즘으로 구현되어 있습니다. kernel은 고정 50ms step으로만 진행하고 벽시계와 난수를 사용하지 않으며, 사칙연산과 제곱근만으로 작성해 두 언어가 같은 IEEE-754 배정밀도 결과를 내도록 보장합니다.

```
Browser                                  Server
  BattleKernel  ──inputs──▶ ledger ──────────▶ battle.Kernel (Go)
  (renders state)                            | replays from published content
  Phaser scene                               ▼
  (pixels only)                            score · stars · progress
```

브라우저는 무슨 일이 일어났는지 보고하지 않고 플레이어가 무엇을 했는지(`{tick, op, …}`)만 제출합니다. 서버는 세션에 고정된 published 콘텐츠를 kernel 입력으로 투영해 digest를 재계산하고, 브라우저가 보낸 digest와 일치할 때만 원장을 재생해 남은 생명·자원·처치·유출·완료 wave·승패를 직접 산출합니다. 두 구현의 동치성은 저장소에 커밋된 replay vector와 콘텐츠 투영 fixture로 CI에서 강제됩니다.

🔐 2. 비밀 저장 및 개인 키 회전

2.1 공급자 비밀: 설치키로 직접 암호화

OIDC client secret과 AI API key는 환경변수 `ENCRYPTION_KEY`의 32바이트 설치키로 직접 AES-256-GCM 암호화하여 DB에 저장합니다. 암호화할 때마다 무작위 nonce를 생성하며, AAD는 `igame:v1`, 저장 형식은 `v1:` 접두사와 nonce·암호문을 담은 Base64URL 문자열입니다. 사용자·테넌트별 DEK를 생성하거나 래핑하는 봉투 암호화 및 Per-User Vault는 구현되어 있지 않습니다.

설치키는 DB와 별도의 비밀 관리소에 보관해야 합니다. 설치키 자동 회전이나 기존 암호문의 재암호화 도구는 제공하지 않으며, 환경변수만 새 키로 바꾸면 이전 키로 암호화한 공급자 비밀을 복호화할 수 없습니다.

2.2 개인 API/MCP 키: 해시 저장과 즉시 회전

개인 API/MCP 키는 암호화해 복구하는 공급자 비밀과 저장 방식이 다릅니다. 서버에는 원문 대신 SHA-256 검증값(`key_hash`)과 prefix·소유자·scope·만료 등 메타데이터를 저장합니다. 원문은 생성 또는 회전의 발급 응답에서 한 번만 제공하며, 이후 조회하거나 복호화해 복구할 수 없습니다.

개인 키의 `rotate`는 기존 키 폐기와 새 키 저장을 같은 트랜잭션으로 처리합니다. 트랜잭션이 성공하면 기존 키는 즉시 폐기되고 새 키 원문을 응답합니다. 구 키와 신 키를 함께 허용하는 자동 유예 기간은 없습니다.

중첩 전환이 필요하면 **별도 새 키 생성 → consumer를 새 키로 전환 → 이전 키 폐기** 순서로 진행합니다. 이 절차는 `rotate`와 구분되며, 활성 키 수와 권한·만료 정책도 확인해야 합니다. 자세한 운영 기준은 저장소의 `docs/security.md`(보안 및 키 관리)에서 「세 가지 키 계층」 절을 찾아 참고하세요.

🔌 3. Model Context Protocol (MCP) 연동

- **규격:** Model Context Protocol (Streamable HTTP)
- **엔드포인트:** `/mcp`
- **세션 유지:** `GET /mcp`은 서버가 먼저 보내는 message가 없는 열린 SSE stream이며 25초마다 keep-alive만 전송합니다. reverse proxy의 stream idle timeout은 이보다 길게 설정해야 합니다
- **제공 도구:**
  - `games_list` / `game_get`: 등록된 게임 목록과 개별 메타데이터 조회
  - `leaderboard_get`: 게임별 개인·부서·팀 랭킹 조회
  - `defense_config_get` / `defense_rankings_get`: Defense Series의 게시된 콘텐츠와 버전 고정 랭킹 조회
  - `profile_get` / `events_list`: 인증된 사용자 프로필과 사내 이벤트 조회
  - `game_session_start` / `score_submit`: 서명된 게임 세션 시작과 점수 제출

- **권한:** API 키로 접근할 때는 도구별로 `games:read`, `rankings:read`, `profile:read`, `sessions:write`, `scores:write` scope를 각각 요구하며 `mcp:access`가 함께 있어야 합니다